



Entorno de desarrollo

Seminario ISGI (S1)



R. Vivó

Entorno de desarrollo

Seminario ISGI (S1)

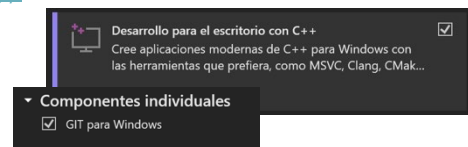
Este seminario explica cómo instalar y poner en funcionamiento el entorno de desarrollo de software gráfico utilizado en la asignatura de Introducción a los Sistemas Gráficos Interactivos del grado en Ingeniería Informática de la ETSINF de la UPV.

Lenguaje y entorno de desarrollo

Se usa el lenguaje C++ bajo el entorno de desarrollo Visual C++ de Microsoft como herramientas sobre las que se va a dar soporte académico. Los laboratorios tienen ya instalado el MS Visual Studio.

En el caso de trabajar con nuestro propio computador, deberemos **instalar el Visual Studio**:

1. Descargar el instalador del Visual Studio Community 202x desde la página de Microsoft <https://visualstudio.microsoft.com/es/vs/community/>
2. En el instalador seleccionar:



El código necesario para trabajar en esta asignatura se encuentra disponible en el repositorio:

https://github.com/RobVivo/Practicas_GL



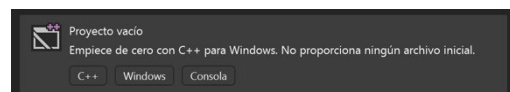
Configuración del entorno y primer proyecto

Para configurar el entorno de trabajo procedemos como nos indica el README del repositorio anterior:

1. Clonar el repositorio en nuestro disco local
2. Indicar dónde se encuentran las bibliotecas dinámicas '*.dll' añadiendo **a la variable 'path' de la cuenta de usuario** el camino hasta la carpeta 'codebase\bin\x86' que se encuentra en el repo que has clonado. Este tutorial te indica cómo:

<https://www.neoguias.com/agregar-directorio-path-windows/>

3. Dentro de la carpeta del repo clonado haz doble clic sobre el fichero con extensión '*.sln'. Se abrirá el Visual Studio. La solución incluye un proyecto *Primer_Proyecto* compilado de ejemplo que debe poder ejecutarse sin problemas si la instalación es correcta
4. Para crear un nuevo proyecto lo agregamos a la solución desde el menú *Archivo->Agregar->Nuevo proyecto*. Elegimos la plantilla '*Proyecto vacío*' y en la siguiente pantalla introducimos el nombre '*Practica1*'.



Hay que cambiar las propiedades del proyecto '*Practica1*' para que reconozca los directorios de cabeceras y bibliotecas. Para ello procedemos así:

5. Seleccionar la plataforma de destino de nuestra aplicación en la barra de herramientas superior
6. Abrir el diálogo de ‘Administrador de propiedades’ con el menú: *Ver -> Otras ventanas -> Administrador de propiedades*. El diálogo aparece como una pestaña más junto a la de ‘Explorador de soluciones’
7. Añadir la hoja de propiedades ‘codebase\OpenGLx86.props’ del repo a la plataforma ‘Release|Win32’ del proyecto en la ventana de ‘Administrador de propiedades’

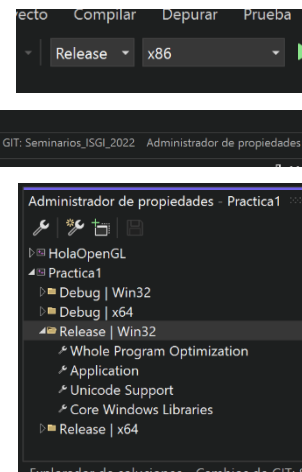
El proyecto ya está configurado y dispuesto para agregar un nuevo fichero de código a ‘Archivos de origen’ en el ‘Explorador de soluciones’

Notas adicionales

Leer la documentación y familiarizarse con el entorno. Una fuente de consulta útil sobre Visual C++ es el MSDN de Microsoft [2]. Sobre asuntos del lenguaje C++ y librerías asociadas puede consultarse también [3].

Es interesante usar la Standard Library para C++ para el manejo de entrada/salida, *strings*, *templates* para estructuras de datos dinámicas, y las librerías clásicas de C.

Es importante incluir antes los ficheros cabecera de la Standard Library como el `cstdio` que los que hacen referencia a OpenGL como `freeglut.h`.



Qué hace y qué no OpenGL

OpenGL es una especificación de la interfaz entre la aplicación de gráficos 3D y el hardware gráfico (API). Consiste en una máquina de estados y algoritmos de procesamiento gráfico que son accesibles a través de decenas de funciones. OpenGL realiza todo el procesamiento gráfico usando el hardware existente para representar en pantalla primitivas gráficas -puntos, líneas y polígonos- definidas en el espacio 3D. Mediante OpenGL es posible configurar el tamaño, posición y orientación de las primitivas, fijar el punto de observación y las condiciones de iluminación, y decidir que aspecto tendrán variando su apariencia.

OpenGL es independiente del hardware y la plataforma usada. Por ello no incluye ni funciones para gestionar ventanas ni funciones para gestionar los eventos de usuario. Estas tareas se encomiendan a librerías, como GLUT, construidas encima de OpenGL y específicas para cada plataforma usada. Así mismo, OpenGL tampoco proporciona funciones para la creación, gestión o interrelación de objetos gráficos 3D más allá de algunas funciones básicas para el modelado de superficies cuádricas y cúbicas incluidas en una librería anexa a OpenGL, la GLU. Para la gestión de escenas (colecciones ordenadas de objetos) se recomiendan herramientas construidas sobre OpenGL como OpenSceneGraph.

Tubería gráfica en OpenGL

Llamamos tubería gráfica a las operaciones que se aplican y que transforman un modelo matemático (informático) de una primitiva 3D en una imagen en pantalla (conjunto de píxeles). El orden de operaciones en OpenGL es éste:

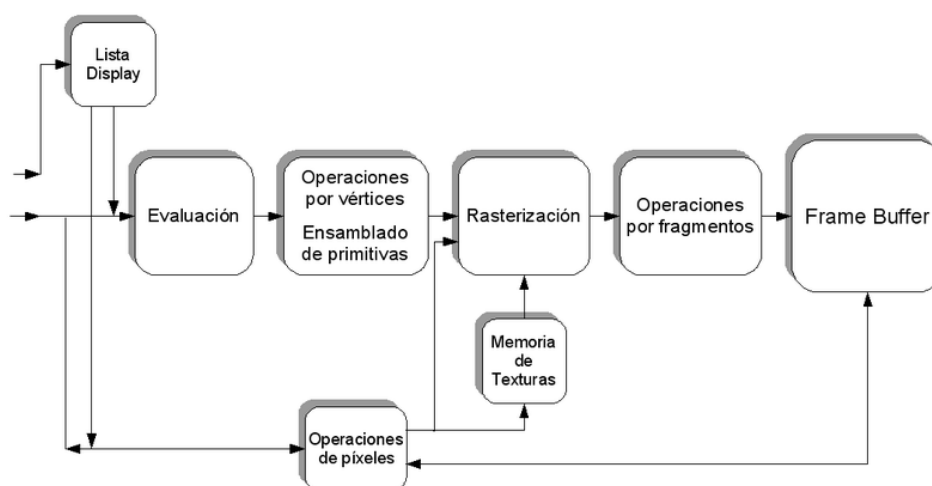


Figura 1. Tubería gráfica fija en OpenGL

Hay dos entidades gráficas principales, los vértices y los fragmentos. Los vértices hacen referencia a un punto en 3D y sus atributos (color, normal, coordenada de textura, etc.), mientras que los fragmentos son elementos pre-píxel, es decir, son posibles píxeles a los que se les asocia información de color, profundidad y transparencia, entre otros. Los fragmentos se obtienen en el proceso de discretización (conversión al *raster*) de las primitivas. Obviamente puede haber dos vértices con las mismas coordenadas, pero con distintos atributos. Lo mismo vale para los fragmentos.

En la tubería fija (figura 1) las operaciones por vértice y fragmento son siempre las mismas y se aplican o no dependiendo de si están habilitadas o deshabilitadas.

Actualmente es posible programar el comportamiento de cada fase de la tubería (excepto la de discretización) mediante lo que se ha dado en llamar “*shaders*”. Este tipo de tubería se denomina dinámica.

Librerías relacionadas con OpenGL

Las librerías imprescindibles que nos van a permitir trabajar con gráficos son las siguientes:

1. OpenGL: Que incluye `gl.h` `glu.h` como ficheros de cabecera y `opengl32.lib` y `opengl32.dll` como librerías. En la instalación del entorno de desarrollo se incluyen todos estos ficheros, por lo que no hay que preocuparse. La librería dinámica `opengl32.dll` se instala en el `System32` habitualmente. Los ficheros de cabecera suelen estar bajo un SDK de Microsoft en archivos de programa. No obstante, puede habilitarse un directorio particular para direccionar todo esto por si no está en el sistema.
2. GLUT¹ (*freeglut*): Que incluye `freeglut.h`, `freeglut32.lib` y `freeglut32.dll`. Los ficheros de ubicarán en directorios convenientes para que sean accesibles tanto en montaje como en ejecución. Por ejemplo, se pueden usar los mismos directorios que para OpenGL. Una fuente desde donde bajarse la librería GLUT es [4]. Es muy recomendable usar *freeglut*, una implementación más reciente, desde el repositorio [5]. Si fuera necesario, es posible generar las librerías usando el *make* apropiado. Otra posibilidad es OpenGLUT que puede encontrarse en [6]. Aunque cualquiera de ellos sirve para la programación básica de gráficos sobre Win32, sólo se dará soporte para *freeglut*.

¹ En adelante, cuando se haga referencia a GLUT, nos estaremos refiriendo a la implementación *freeglut*

Con la instalación de estas dos tecnologías es suficiente para generar gráficos por computador en el paradigma de tubería fija y con interactividad limitada al uso del ratón, teclas y menús de pop-up sin otros controles más complejos como botones, cajas de texto, etc., típicos de cualquier interfaz. Si se quiere ampliar las capacidades de programación, por ejemplo, usando modo retenido o tubería dinámica, o se quiere mejorar la interfaz con controles mejores, los paquetes que se recomienda instalar son:

1. GLEW: Esta librería hace un envoltorio a la `gl.h` para poder usar las extensiones más allá de la versión 1.2 de OpenGL que las tarjetas gráficas incorporan hoy día. Para instalar la GLEW haremos lo mismo. Descargamos el paquete desde [7] para Win 32, copiamos los ficheros de cabecera a nuestro directorio de *includes* (`include/GL`), los ficheros *lib* al directorio `LIB` y la *dll* al *path* de ejecución. GLEW ofrece dos librerías la `glew32.lib` y la `glew32s.lib` que corresponden a la versión dinámica y estática respectivamente.
2. GLUT: Nos ofrece controles avanzados construidos sobre OpenGL, por lo que su integración es fácil. Se puede descargar desde [8].

Existen otras posibilidades para la construcción de aplicaciones interactivas con una interfaz más cuidada como son las basadas en la tecnología Qt que exceden el ámbito de conocimiento de este seminario.

Hay documentación abundante sobre cada una de las librerías anteriores en las direcciones de descarga. Se recomienda consultarla.

Sintaxis en OpenGL y GLUT

OpenGL y GLUT son librerías cuyo API consiste en:

1. Funciones que comienzan con `gl`, `glu`, `glut` según correspondan al API de OpenGL, GLU o GLUT respectivamente. El resto del nombre la función sigue el convenio de iniciales por ejemplo, `glutInitDisplayMode()`.
2. Tipos predefinidos, por ejemplo, `GLfloat`.
3. Constantes que comienzan por `GL_` o `GLUT_` según la librería. El nombre de la constante es en mayúsculas y se separa por guiones bajos si consta de más de una palabra, por ejemplo, `GL_COLOR_BUFFER_BIT`.
4. El API de OpenGL está orientado al lenguaje C. Por ello no existe polimorfismo en las funciones según el número o tipo de parámetros. Para subsanar esto, la misma función con diferente tipo o número en los parámetros tiene un sufijo diferente, por ejemplo para situar un punto puede usarse según el caso `glVertex2f(float x, float y)`, `glVertex3f(float x, float y, float z)` o `glVertex3i(int x, int y, int z)` entre otras.

Hola mundo

Vamos a generar la primera aplicación de gráficos con OpenGL, GLUT y Visual C++. El proceso para crear un nuevo proyecto ya se ha descrito en los puntos anteriores. Ahora nos centramos en la implementación del código fuente sobre el fichero que agregamos a ‘Archivos de origen’.

Para ello escribiremos el código siguiente:

```

#include <iostream>
#include <gl\freeglut.h>
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glFlush();
}
void reshape(GLint w, GLint h)
{
}
void main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(200,200);
    glutCreateWindow("Hola Mundo");
    std::cout << "Hola Mundo running" << std::endl;
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMainLoop();
}

```

Este es el código más sencillo que podemos escribir y que nos servirá de plantilla para todos los programas sobre OpenGL. Para generar el ejecutable usamos la opción del menú “Compilar->Compilar HolaMundo” si así hemos llamado a nuestro proyecto. Esto debería compilar y montar sin errores a menos que no se hayan incluido bien las librerías. Para ejecutar, pulsar sobre el triángulo verde . La ejecución produce dos ventanas, la **consola** por donde salen los mensajes “cout” y la **ventana de dibujo** en negro pues no se ha dibujado nada. Observar las dimensiones de la ventana de dibujo y su título.

Observar el orden de inclusión de los ficheros de cabecera así como la función que cumplen.

Las funciones `display()` y `reshape(int,int)` se conocen como **callbacks** y son las que se ejecutan cuando se dispara un evento de esa clase. Es necesario registrarlas en GLUT antes del inicio del bucle de atención a eventos. El manejador de eventos “MainLoop” es el encargado de recoger los eventos de usuario y llamar a la función que corresponda a su tratamiento.

Nuevas funciones y constantes

Sigue una pequeña referencia de las funciones y constantes utilizadas. Mientras no se diga lo contrario todas las funciones son `void`.

GLUT

glutInit()

Sirve para inicializar la GLUT y debe llamarse antes de cualquier llamada. Puede procesar los parámetros del main pero es dependiente de la plataforma. Lo normal es llamarla sin parámetros.

glutInitDisplayMode(unsigned int formatoBuffers)

Configura la memoria gráfica según los buffers que se vayan a utilizar. Por defecto, si no se llama, la configuración es de un buffer de color con canal alfa en modo RGB. Corresponde a usar como parámetros `GLUT_RGBA|GLUT_SINGLE`. El parámetro es una máscara de bits por combinación or de constantes predefinidas. Se puede usar `GLUT_DOUBLE` en vez de `GLUT_SINGLE` para tener un buffer trasero además del buffer frontal (*front y back buffers*). `GLUT_DEPTH` para el uso del *Z-buffer*, `GLUT_STENCIL` para el uso del *stencil buffer*, entre otros. La llamada debe hacerse antes de la creación de la ventana.

glutInitWindowSize(int ancho, int alto)

Determina el tamaño de la ventana de dibujo que se va a crear. Los parámetros indican el alto y el ancho en píxeles.

int glutCreateWindow(char titulo)*

Crea la ventana, sin mostrarla todavía, y configura todas sus características como tamaño, posición y formato de buffers. El parámetro hace referencia al título que aparece en la barra superior de la ventana (barra de título) si es que existe con el manejador de ventanas usado. No se puede dibujar todavía sobre la ventana hasta que no se ponga en marcha el bucle de eventos con `glutMainLoop()`. La función devuelve un identificador único (manejador de la ventana). El identificador sirve para referirse a esa ventana en concreto, pasarle el foco, por ejemplo, cuando se usa multiventana. Cada ventana mantiene su propio contexto de dibujo que ha quedado configurado con la llamada a esta función.

glutDisplayFunc()

Permite registrar la función que atenderá el evento de “*display*”. Este evento se produce cuando el contenido de la ventana de dibujo cambia por cualquier razón. La causa más frecuente es que se fuerce desde dentro de la aplicación el redibujo porque algo ha cambiado (por ejemplo, en una animación). La forma de registrar la función de atención, llamada *callback*, es pasar su nombre como parámetro p.e. `glutDisplayFunc(onDisplay)` siendo `onDisplay()` la función que se activará cuando se procese el evento de redibujo.

glutReshapeFunc()

Como la anterior permite registrar la *callback* que atiende al evento de redimensión de la ventana de dibujo, por ejemplo, cuando “estiramos” la ventana con el ratón. El evento de redimensión también se lanza cuando se crea la ventana.

glutMainLoop()

Esta función activa el bucle de atención de eventos. Cada evento que se origine en el sistema (ratón, teclado, etc.), o sea generado por la propia aplicación (p.e. redibujo), llega a una cola de eventos que se procesa en secuencia. Todos los eventos que tengan asociada una *callback* por el registro anterior son atendidos por el manejador activando la *callback*, a la que se le pasan los parámetros del evento si éste los tuviera.

GLUT_SINGLE

Constante que indica la selección de un único *buffer* de dibujo.

GLUT_RGB

Constante que indica la selección del modo de color rojo, verde, azul.

OpenGL

glClear()

Orden de borrado de cualquier buffer. Pueden indicarse varios simultáneamente mediante constantes que actúan como máscaras OR.

glFlush()

Orden de volcado a la tarjeta gráfica de todas las ordenes de dibujo pendientes de emitir. Con esto nos aseguramos que el dibujo se ha completado antes de proseguir.

GL_COLOR_BUFFER_BIT

Constante que se refiere al *buffer* de dibujo y que se inserta como parámetro en la función anterior `glClear()`.

Bibliografía

- [1] «Visual Studio Community» [En línea, verano 2024]. Available: <https://visualstudio.microsoft.com/es/vs/community/>
- [2] «Documentación MS Visual C++» [En línea, verano 2024]. Available: <https://learn.microsoft.com/es-es/cpp/?view=msvc-170>
- [3] «Documentación C++» [En línea, verano 2024]. Available: <http://cplusplus.com/>.
- [4] «Visual Studio con OpenGL y GLUT,» [En línea]. Available: ~~<https://polimedia.upv.es/visor/?id=d73df12b-45b7-7a44-9cd9-1ba0de2a5e41>~~.
- [5] «GLUT de Nate Robins,» [En línea]. Available: <http://user.xmission.com/~nate/glut.html>. - deprecated-
- [6] «Freeglut,» [En línea, verano 2004]. Available: <http://sourceforge.net/projects/freeglut/>.
- [7] «OpenGLUT,» [En línea]. Available: <http://sourceforge.net/projects/openglut.-deprecated->
- [8] «GLEW: OpenGL Extension Wrangler Library,» [En línea, verano 2004]. Available: <http://glew.sourceforge.net/>.
- [9] «GLUI: User Interface Library,» [En línea, verano 2004]. Available: <https://github.com/libglui/glui>.

```

/*****
  HolaGL.cpp

  Esqueleto basico de un programa en OpenGL

  @author rvivo@upv.es
  *****/
#define PROYECTO "ISGI::S1E01::Hola Mundo"

#include <iostream>                                // Biblioteca de entrada salida
#include <codebase.h>                              // Biblioteca grafica

void display()
// Funcion de atencion al dibujo
{
    glClear(GL_COLOR_BUFFER_BIT);
    glFlush();
}

void reshape(GLint w, GLint h)
// Funcion de atencion al redimensionamiento
{
}

void main(int argc, char** argv)
// Programa principal
{
    glutInit(&argc, argv);                        // Inicializacion de GLUT
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);    // Alta de buffers a usar
    glutInitWindowSize(200,200);                 // Tamanyo inicial de la ventana
    glutCreateWindow(PROYECTO);                   // Creacion de la ventana con su titulo
    std::cout << PROYECTO << " running" << std::endl; // Mensaje por consola
    glutDisplayFunc(display);                     // Alta de la funcion de atencion a display
    glutReshapeFunc(reshape);                     // Alta de la funcion de atencion a reshape
    glutMainLoop();                               // Puesta en marcha del programa
}

```